

Document number: P3896R0
Date: 2025-10-30
Audience: EWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>

Design goals for a contract support facility

Given the number of "compromise", "redesign" or "fix" proposals that surfaced recently regarding the contract support facility, we feel it necessary to remind one of its main design goals.

The goal

The initial idea for encoding/embracing the function contracts in C++, back in 2014, if not earlier, most prominently expressed in [\[N4319\]](#), was that the programmer puts a single annotation about the function contract, and this enables two kinds of different correctness-related tools:

1. Optional runtime evaluation, as in C's macro `assert()`.
2. Correctness-checking, static analysis, ideally local.

It is two different kinds of tools, but they consume the very same annotations. For this to work, the typical usage of both these features needs to be changed/compromised from its natural shape:

1. The assertions need to appear in function declarations, which is unusual for run-time checking.
2. The assertions are in the form of C++ expressions convertible to `bool`, which is unusual for annotations targeting static analysis.

(Although, in a full product — as opposed to MVP — the predicates would be extended from ordinary C++ expressions, to more constructs, allowing to cover the full spectrum of static analysis requirements. Those would be either axioms from [\[P0542R5\]](#) or compiler intrinsics.)

This idea follows one of the most important design guidelines of the C++ Evolution: if you need to add a new feature and complicate the language, make sure that it solves more than one problem. This approach has been followed in [\[P0542R5\]](#) and is now followed in [\[P2900R14\]](#).

In this sense, the proposals do not just standardize the existing practice, but try to take it to the next level: create a synergy between optional runtime checking and static analysis. This comes with new challenges that "just runtime checking" approaches do not need to address.

The main value proposition to the programmers is this: you just express your precondition once, and enjoy multiple ways of testing correctness based on it.

The designs in [\[N4319\]](#), [\[P0542R5\]](#), [\[P2900R14\]](#) did not take into account another technique that has been communicated to the authors only recently (2024): where a library gives a non-disableable *guarantee* that it will terminate the program upon any violation of the expressed precondition. (While the design in [\[P2900R14\]](#) has been demonstrated to be extensible enough to evolve to cover this case, on itself, at this point it does not support it directly.) This technique is borderline when it comes to fitting the model: on the one hand, the library author expresses a precondition; on the other they give a guarantee as to what is going to happen, making the behavior part of the contract, and reducing the pressure on the users to make sure that the declared condition must be satisfied.

The common part of the three tools is expressing in the language what the contract is and who — caller versus callee — is responsible for guaranteeing that they hold. These expressed conditions are expected to be satisfied by means other than the contract assertions: by the regular control flow, by input validation. Contract assertions are only for re-asserting something that is understood to have been already taken care of by the programmer.

What will not work

In this context, the recent compromise proposals that say "just offer `contract_assert`" with a guaranteed termination, are not steps toward achieving the above goal. They are just a standalone single-purpose feature. If I were to be offered only this, I would ask a question: how is the feature better than my home-made macro? What do I get that I do not already have today? If this is only about having a global standardized callback (the contract violation handler), then maybe just standardize this: the interface for the handler. Simply adapt [\[P3290R3\]](#). No `pre`, `post` or `contract_assert` keywords in any flavor are necessary. For convenience, offer an additional Standard Library function:

```
void enforce(bool predicate_result,
             const source_location& loc = source_location::current());
```

that would be called like this:

```
std::enforce(x > 0);
```

This would do nothing if the parameter passed is `true` and call the handler and terminate when the parameter is `false`. Just don't call it "contract".

Another thing that would not work is to provide a set of small orthogonal features — preambles and postambles, function call decorators, ADL-control, lazy function evaluation, semantic control — and allow every user to compose their own "assertion". This allows a fine-grained control of the runtime checking, but prevents writing tools for effective static analysis based on correctness annotations (contract assertions). For these to work there has to be **the one**, standardized, universally recognized notation for these annotations.

In this context, by "static analysis" we mean the kind that tries to determine if there are inputs to the program that can cause any assertion to be violated. Here the assertions are treated as "annotations" and their evaluation semantics are irrelevant.

References

- [N4319] — Gabriel Dos Reis, Shuvendu Lahiri, Francesco Logozzo, Thomas Ball, Jared Parsons, "Contracts for C++: What Are the Choices?", <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4319.pdf>.
- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++", <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>.
- [P2900R14] — Joshua Berne, Timur Doumler, Andrzej Krzemiński et al., "Contracts for C++", <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2900r14.pdf>.
- [P3290R3] — Joshua Berne, Timur Doumler, John Lakos, "Integrating Existing Assertions with Contracts", <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3290r3.pdf>.